

3

资源管理

Resource Management

所谓资源就是，一旦用了它，将来必须还给系统。如果不这样，糟糕的事情就会发生。C++ 程序中最常使用的资源就是动态分配内存（如果你分配内存却从来不曾归还它，会导致内存泄漏），但内存只是你必须管理的众多资源之一。其他常见的资源还包括文件描述器（file descriptors）、互斥锁（mutex locks）、图形界面中的字型 and 笔刷、数据库连接、以及网络 sockets。不论哪一种资源，重要的是，当你不再使用它时，必须将它还给系统。

尝试在任何运用情况下都确保以上所言，是件困难的事，但当你考虑到异常、函数内多重回传路径、程序维护员改动软件却没能充分理解随之而来的冲击，态势就很明显了：资源管理的特殊手段还不很充分够用。

本章一开始是一个直接而易懂且基于对象（object-based）的资源管理办法，建立在 C++ 对构造函数、析构函数、*copying* 函数的基础上。经验显示，经过训练后严守这些做法，可以几乎消除资源管理问题。然后本章的某些条款将专门用来对付内存管理。这些排列在后的专属条款弥补了先前一般化条款的不足，因为管理内存的那个对象必须知道如何适当而正确地工作。

条款 13: 以对象管理资源

Use objects to manage resources.

假设我们使用一个用来塑模投资行为（例如股票、债券等等）的程序库，其中各式各样的投资类型继承自一个 root class `Investment::`

```
class Investment { ... };           // “投资类型” 继承体系中的 root class
```

进一步假设，这个程序库系通过一个工厂函数（factory function，见条款 7）供应我们某特定的 Investment 对象：

```
Investment* createInvestment(); //返回指针，指向 Investment 继承体系内
                                //的动态分配对象。调用者有责任删除它。
                                //这里为了简化，刻意不写参数。
```

一如以上注释所言，createInvestment 的调用端使用了函数返回的对象后，有责任删除之。现在考虑有个 f 函数履行了这个责任：

```
void f()
{
    Investment* pInv = createInvestment(); //调用 factory 函数
    ...
    delete pInv; //释放 pInv 所指对象
}
```

这看起来妥当，但若干情况下 f 可能无法删除它得自 createInvestment 的投资对象——或许因为 “...” 区域内的一个过早的 return 语句。如果这样一个 return 被执行起来，控制流就绝不会触及 delete 语句。类似情况发生在对 createInvestment 的使用及 delete 动作位于某循环内，而该循环由于某个 continue 或 goto 语句过早退出。最后一种可能是 “...” 区域内的语句抛出异常，果真如此控制流将再次不会幸临 delete。无论 delete 如何被略过去，我们泄漏的不只是内含投资对象的那块内存，还包括那些投资对象所保存的任何资源。

当然啦，谨慎地编写程序可以防止这一类错误，但你必须想想，代码可能会在时间渐渐过去后被修改。一旦软件开始接受维护，可能会有某些人添加 return 语句或 continue 语句而未能全然领悟它对函数的资源管理策略造成的后果。更糟的是 f 的 “...” 区域有可能调用一个“过去从未抛出异常，却在被‘改善’之后开始那么做”的函数。因此单纯倚赖“f 总是会执行其 delete 语句”是行不通的。

为确保 createInvestment 返回的资源总是被释放，我们需要将资源放进对象内，当控制流离开 f，该对象的析构函数会自动释放那些资源。实际上这正是隐身于本条款背后的半边想法：把资源放进对象内，我们便可倚赖 C++ 的“析构函数自动调用机制”确保资源被释放。（稍后讨论另半边想法。）

许多资源被动态分配于 `heap` 内而后被用于单一区块或函数内。它们应该在控制流离开那个区块或函数时被释放。标准程序库提供的 `auto_ptr` 正是针对这种形势而设计的特制产品。`auto_ptr` 是个“类指针 (pointer-like) 对象”，也就是所谓“智能指针”，其析构函数自动对其所指对象调用 `delete`。下面示范如何使用 `auto_ptr` 以避免 `f` 函数潜在的资源泄漏可能性：

```
void f( )
{
    std::auto_ptr<Investment> pInv(createInvestment( ));
                                   //调用 factory 函数
    ...                             //一如以往地使用 pInv
}                                   //经由 auto_ptr 的析构函数自动删除 pInv
```

这个简单的例子示范“以对象管理资源”的两个关键想法：

- **获得资源后立刻放进管理对象 (managing object) 内。** 以上代码中 `createInvestment` 返回的资源被当做其管理者 `auto_ptr` 的初值。实际上“以对象管理资源”的观念常被称为“资源取得时机便是初始化时机” (*Resource Acquisition Is Initialization; RAII*)，因为我们几乎总是在获得一笔资源后于同一语句内以它初始化某个管理对象。有时候获得的资源被拿来赋值 (而非初始化) 某个管理对象，但不论哪一种做法，每一笔资源都在获得的同时立刻被放进管理对象中。
- **管理对象 (managing object) 运用析构函数确保资源被释放。** 不论控制流如何离开区块，一旦对象被销毁 (例如当对象离开作用域) 其析构函数自然会被自动调用，于是资源被释放。如果资源释放动作可能导致抛出异常，事情变得有点棘手，但条款 8 已经能够解决这个问题，所以这里我们也就多操心了。

由于 `auto_ptr` 被销毁时会自动删除它所指之物，所以一定要注意别让多个 `auto_ptr` 同时指向同一对象。如果真是那样，对象会被删除一次以上，而那会使你的程序搭上驶向“未定义行为”的快速列车上。为了预防这个问题，`auto_ptr`s 有一个不寻常的性质：若通过 `copy` 构造函数或 `copy assignment` 操作符复制它们，它们会变成 `null`，而复制所得的指针将取得资源的唯一拥有权！

```

std::auto_ptr<Investment>
    pInv1(createInvestment());           //pInv1 指向
                                         // createInvestment 返回物.
std::auto_ptr<Investment> pInv2(pInv1);  //现在 pInv2 指向对象,
                                         // pInv1 被设为 null.
pInv1 = pInv2;                          //现在 pInv1 指向对象,
                                         // pInv2 被设为 null.

```

这一诡异的复制行为，复加上其底层条件：“受 `auto_ptr` 管理的资源必须绝对没有一个以上的 `auto_ptr` 同时指向它”，意味 `auto_ptr` 并非管理动态分配资源的神兵利器。举个例子，STL 容器要求其元素发挥“正常的”复制行为，因此这些容器容不得 `auto_ptr`。

`auto_ptr` 的替代方案是“引用计数型智慧指针”(reference-counting smart pointer; RCSP)。所谓 RCSP 也是个智能指针，持续追踪共有多少对象指向某笔资源，并在无人指向它时自动删除该资源。RCSPs 提供的行为类似垃圾回收 (garbage collection)，不同的是 RCSPs 无法打破环状引用 (cycles of references，例如两个其实已经没被使用的对象彼此互指，因而好像还处在“被使用”状态)。

TR1 的 `tr1::shared_ptr` (见条款 54) 就是个 RCSP，所以你可以这么写 `f`：

```

void f()
{
    ...
    std::tr1::shared_ptr<Investment>
        pInv(createInvestment());           //调用 factory 函数.
    ...                                     //使用 pInv 一如以往.
}                                           //经由 shared_ptr 析构函数自动删除 pInv

```

这段代码看起来几乎和使用 `auto_ptr` 的那个版本相同，但 `shared_ptr` 的复制行为正常多了：

```

void f()
{
    ...
    std::tr1::shared_ptr<Investment>
        pInv1(createInvestment());           //pInv1 指向
                                         //createInvestment 返回物.
    std::tr1::shared_ptr<Investment>
        pInv2(pInv1);                       //pInv1 和 pInv2 指向同一个对象.
    pInv1 = pInv2;                          //同上，无任何改变.
    ...
}                                           //pInv1 和 pInv2 被销毁,
                                         //它们所指的对象也就被自动销毁.

```

由于 `tr1::shared_ptr` 的复制行为“一如预期”，它们可被用于 STL 容器以及其他“`auto_ptr` 之非正统复制行为并不适用”的语境上。

尽管如此，可别误会了，本条款并不专门针对 `auto_ptr`, `tr1::shared_ptr` 或任何其他智能指针，而只是强调“以对象管理资源”的重要性，`auto_ptr` 和 `tr1::shared_ptr` 只不过是实际例子。如果想知道 `tr1::shared_ptr` 的更多信息，请看条款 14, 18 和 54。

`auto_ptr` 和 `tr1::shared_ptr` 两者都在其析构函数内做 `delete` 而不是 `delete[]` 动作（条款 16 对两者的不同有些描述）。那意味在动态分配而得的 `array` 身上使用 `auto_ptr` 或 `tr1::shared_ptr` 是个馊主意。尽管如此，可叹的是，那么做仍能通过编译：

```
std::auto_ptr<std::string>           //馊主意！会上错误的
aps(new std::string[10]);             // delete 形式。
std::tr1::shared_ptr<int> spi(new int[1024]); //相同问题。
```

你或许会惊讶地发现，并没有特别针对“C++ 动态分配数组”而设计的类似 `auto_ptr` 或 `tr1::shared_ptr` 那样的东西，甚至 TR1 中也没有。那是因为 `vector` 和 `string` 几乎总是可以取代动态分配而得的数组。如果你还是认为拥有针对数组而设计、类似 `auto_ptr` 和 `tr1::shared_ptr` 那样的 `classes` 较好，看看 Boost 吧（见条款 55）。在那儿你会很高兴地发现 `boost::scoped_array` 和 `boost::shared_array` `classes`，它们都提供你要的行为。

本条款也建议，如果你打算手工释放资源（例如使用 `delete` 而非使用一个资源管理类; `resource-managing class`），容易发生某些错误。罐装式的资源管理类如 `auto_ptr` 和 `tr1::shared_ptr` 往往比较能够轻松遵循本条款忠告，但有时候你所使用的资源是目前这些预制式 `classes` 无法妥善管理的。既然如此就需要精巧制作你自己的资源管理类。那并不是非常困难，但的确涉及若干你需要考虑的细节。那些考虑形成了条款 14 和条款 15 的标题。

作为最后批注，我必须指出，`createInvestment` 返回的“未加工指针”（`raw pointer`）简直是对资源泄漏的一个死亡邀约，因为调用者极易在这个指针身上忘记调用 `delete`。（即使他们使用 `auto_ptr` 或 `tr1::shared_ptr` 来执行 `delete`，他们首先必须记得将 `createInvestment` 的返回值存储于智能指针对象内。）为与此问题搏斗，首先需要对 `createInvestment` 进行接口修改，那是条款 18 面对的事。

请记住

- 为防止资源泄漏，请使用 RAII 对象，它们在构造函数中获得资源并在析构函数中释放资源。
- 两个常被使用的 RAII classes 分别是 `tr1::shared_ptr` 和 `auto_ptr`。前者通常是较佳选择，因为其 *copy* 行为比较直观。若选择 `auto_ptr`，复制动作会使它（被复制物）指向 `null`。

条款 14：在资源管理类中小心 *coping* 行为

Think carefully about copying behavior in resource-managing classes.

条款 13 导入这样的观念：“资源取得时机便是初始化时机”（*Resource Acquisition Is Initialization*; RAII），并以此作为“资源管理类”的脊柱，也描述了 `auto_ptr` 和 `tr1::shared_ptr` 如何将这个观念表现在 `heap-based` 资源上。然而并非所有资源都是 `heap-based`，对那种资源而言，像 `auto_ptr` 和 `tr1::shared_ptr` 这样的智能指针往往不适合作为资源掌管者（`resource handlers`）。既然如此，有可能偶而你会发现，你需要建立自己的资源管理类。

例如，假设我们使用 C API 函数处理类型为 `Mutex` 的互斥器对象（`mutex objects`），共有 `lock` 和 `unlock` 两函数可用：

```
void lock (Mutex* pm);           //锁定 pm 所指的互斥器。
void unlock (Mutex* pm);        //将互斥器解除锁定。
```

为确保绝不会忘记将一个被锁住的 `Mutex` 解锁，你可能会希望建立一个 class 用来管理机锁。这样的 class 的基本结构由 RAII 守则支配，也就是“资源在构造期间获得，在析构期间释放”：

```
class Lock {
public:
    explicit Lock (Mutex* pm)
        : mutexPtr (pm)
    { lock (mutexPtr); }           //获得资源
```